

SIMATSENGINEERING

ASSESSMENTTOOL1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 25 *100*

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

I G.THARAKNATHREDDY(192472032) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

G. Tharak
Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

- of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.
4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
 5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

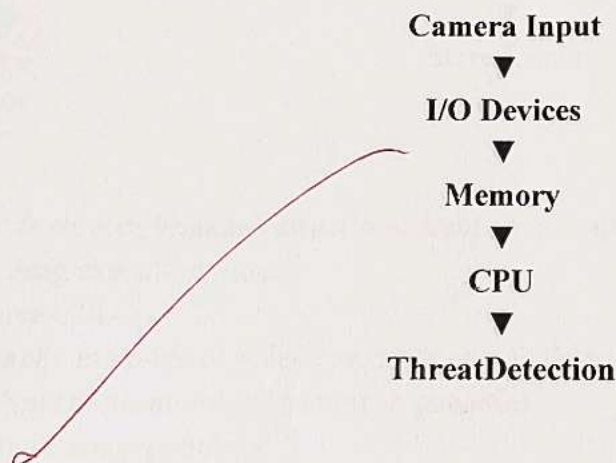
Q1. Smart City Surveillance System

Answer:

A smart city surveillance system continuously collects video data from multiple cameras. The data is transferred between **I/O devices, memory, and CPU** for processing. During peak hours, thousands of video frames are processed simultaneously, increasing system load and causing delays.

Interaction of CPU, Memory, and I/O

- **I/O devices** (cameras) capture video and send it to the computer.
- **Memory (RAM)** temporarily stores the video frames.
- **CPU** fetches instructions and data from memory, processes them, and identifies threats.
- If the bus is busy or memory access is slow, the CPU remains idle, reducing overall performance.
- **Flow chart:**



Bus Structures

Single Bus Architecture

- One bus is shared by CPU, memory, and I/O devices.
- Only one data transfer occurs at a time.
- Causes congestion and increases waiting time.

Multi-Bus Architecture

- Uses separate buses for CPU, memory, and I/O communication.
- Multiple transfers can occur simultaneously.

- Reduces bottlenecks and improves system speed.
- Improving Instruction Execution**
- Use instruction pipelining to execute multiple instructions simultaneously.
 - Increase cache memory to reduce RAM access time.
 - Implement DMA (Direct Memory Access) for faster I/O transfer.
 - Use faster processors and optimize scheduling.

Conclusion

A multi-bus system, pipelining, cache memory, and DMA significantly improve surveillance performance, reduce lag, and enable faster real-time threat detection.

Q2. Real-Time Stock Trading Application

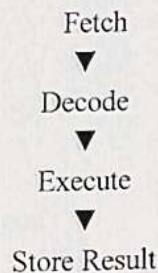
Answer:

A real-time stock trading application executes millions of instructions every second. During heavy trading, users experience delays because the processor has a **high CPI (Cycles Per Instruction)** due to frequent memory access and branch instructions.

Fetch-Decode-Execute Cycle

1. **Fetch:** CPU fetches the instruction from memory.
2. **Decode:** CPU interprets the instruction.
3. **Execute:** CPU performs the required operation.
4. **Store:** The result is written back to memory or registers.

Flow chart:



If memory access is slow or branch instructions occur frequently, the CPU spends more clock cycles on each instruction, increasing execution time.

Methods to Reduce CPI

- Increase **cache memory** to reduce memory access delay.
- Use **branch prediction** to minimize branch penalties.
- Apply **instruction pipelining**.
- Use **superscalar architecture** to execute multiple instructions together.
- Optimize software to reduce unnecessary instructions.
- Increase processor clock frequency where possible.

Conclusion

Reducing CPI improves CPU efficiency, decreases execution time, and ensures faster transaction processing during high workloads.

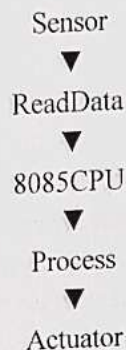
Q3.8085 Embedded Temperature Control Unit Answer:

The 8085 microprocessor controls industrial temperature systems by reading sensor data and sending control signals to actuators. Correct addressing modes are essential for accurate data access.

Addressing Modes

- **Immediate Addressing:** Data is directly available in the instruction.
- **Register Addressing:** Uses CPU registers for faster processing.
- **Direct Addressing:** The memory address is specified explicitly.
- **Register Indirect Addressing:** Memory address is stored in a register pair.

Flow chart:



Causes of Incorrect Readings

- Wrong addressing mode selected.
- Incorrect memory address.
- Improper sensor calibration.
- Programming errors in data transfer.
- Noise or hardware faults affecting sensor input.

Effective Use of 8085 Instruction Set

- Use instructions like MOV, MVI, LDA, STA, LXI, IN, and OUT correctly.
- Validate sensor readings before processing.
- Store data in the correct memory locations.
- Test the program under different operating conditions.

Conclusion

Proper use of addressing modes and the 8085 instruction set ensures accurate temperature readings, reliable control, and stable industrial operation.

Q4.8086 Segmented Memory Answer:

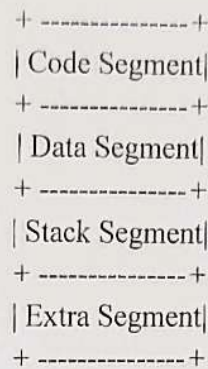
The 8086 microprocessor uses segmented memory to efficiently manage programs and data. Memory is divided into logical segments, allowing multiple programs to execute efficiently.

Memory Segments

- **Code Segment (CS):** Stores program instructions.
- **Data Segment (DS):** Stores variables and data.
- **Stack Segment (SS):** Stores return addresses and temporary data.

- **Extra Segment (ES):** Stores additional data for string operations.

Flow chart:



Problems Due to Improper Segment Handling

- Incorrect segment registers access wrong memory locations.
- Stack overflow occurs when too many values are pushed without popping.
- Program crashes because of invalid memory references.
- Data corruption occurs due to overlapping segments.

Proper Instruction Execution

- Initialize all segment registers correctly.
- Use correct addressing modes.
- Balance every **PUSH** instruction with a corresponding **POP**.
- Monitor stack size during execution.
- Debug memory references carefully.

Conclusion

Proper segmentation, correct addressing, and balanced stack operations ensure reliable execution of multiple programs without memory errors.

Q5. Number System Conversion

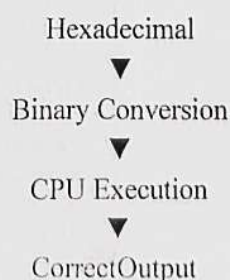
Answer:

Computers process data in **binary**, while programmers often use **hexadecimal** because it is compact and easier to understand. Accurate conversion between hexadecimal and binary is essential for debugging and communication systems.

Importance of Conversion

- Simplifies binary representation.
- Used in memory addresses and machine instructions.
- One hexadecimal digit represents **4 binary bits**.
- Makes debugging and data analysis easier.

Flow chart:



Effects of Conversion Errors

- Wrong instruction execution.
- Incorrect memory addressing.
- Packet transmission errors.
- Data corruption and communication failure.
- Reduced system reliability.

Role of Efficient CPU Design

- Faster CPUs execute instructions more accurately.
- Error detection and correction mechanisms improve reliability.
- Cache memory reduces execution delays.
- Benchmarking evaluates CPU speed, throughput, and efficiency under different workloads.

Conclusion

Correct hexadecimal-to-binary conversion, efficient CPU architecture, and benchmarking help improve system accuracy, detect errors early, and ensure reliable real-time communication.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	4	3	3	2	15
Q2	3	3	3	3	2	14
Q3	2	3	4	3	2	14
Q4	3	3	4	3	1.5	14.5
Q5	3	4	3	3	1.5	14.5

72

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

Problem-Solving Approach	20	Logical, ⁴ well-structured, and efficient approach	Mostly logical ³ approach with minor gaps	Basic approach ² with some inconsistencies	Illogical or unclear ¹ approach
Accuracy of Solution	20	Completely correct ⁴ solution with proper justification	Mostly correct ³ with minor errors	Partially ² correct solution	Incorrect or incomplete ¹ solution
Clarity	10	Clear, ² well-organized, and properly explained solution	Understandable ^{1.5} with minor clarity issues	Limited clarity ¹ and explanation	Poorly presented ⁰ and difficult to understand